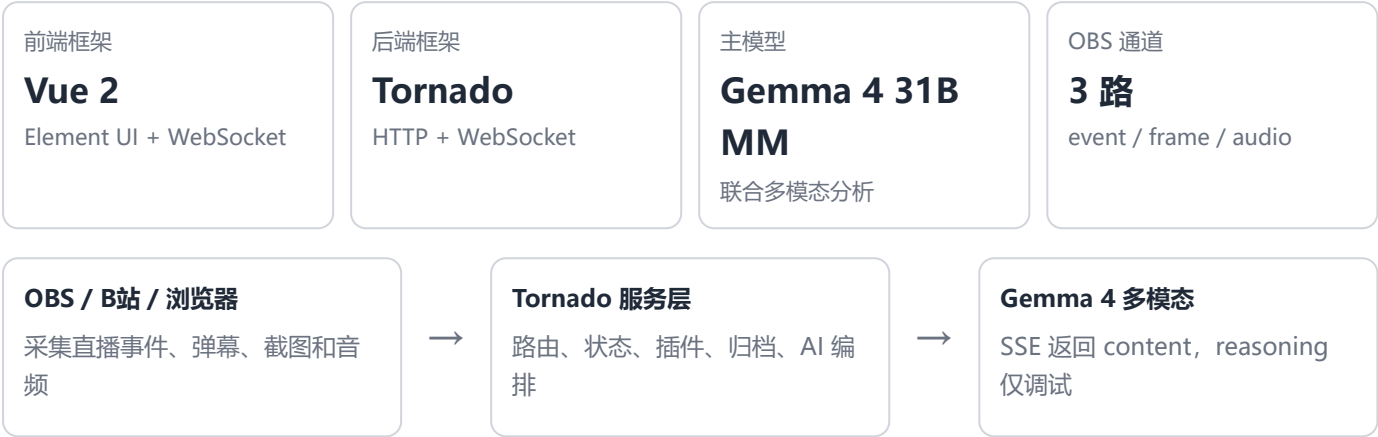


# 客户端技术架构报告

范围：当前仓库的客户端、后端协作、Agent、OBS 插件、跨软件通信和工具栈。依据实际文件包括 `main.py`、`api/qwen_multimodal.py`、`services/obs_bridge.py`、`frontend/src/views/Room.vue`、`obs-blivechat-bridge/src/obs-blivechat-bridge.cpp`。



## 一、模型选型理由

| 模型/策略                      | 使用位置              | 选择理由                                               |
|----------------------------|-------------------|----------------------------------------------------|
| <code>gemma4-31b-mm</code> | 联合心理/语音/画面分析与视觉场景 | 31B 规模给足复杂推理和多模态融合能力，适合把截图、双路音频和弹幕统计合并成一个结构化 JSON。 |
| <code>gemma4-e4b-mm</code> | ASR/轻量语音观察        | E4B 规格更轻，用于单独语音转写和声音分类，降低延迟和资源占用。                  |
| 规则分析 fallback              | 无模型或非结构化返回时       | 保障 UI 不空白；当模型返回英文 reasoning 或非 JSON 时，只展示本地中文兜底结果。 |

选择 `gemma4-31b-mm` 的核心理由是联合分析任务不是单纯 OCR 或 ASR，而是需要同时理解截图、弹幕统计、麦克风音频、桌面音频和直播语境。31B 规模提供更强的跨模态归纳和结构化输出能力。对单独 ASR/语音观察，`gemma4-e4b-mm` 更轻，适合降低成本与延迟。

当前客户端策略是：业务 UI 只展示 `delta.content` 或可解析 JSON；`delta.reasoning` 只写后端 debug 日志，避免英文推理过程污染用户界面。

## 二、系统分层

| 层级            | 关键文件                                                                                                                               | 职责                                                                  |
|---------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Vue 客户端       | <code>frontend/src/views/Home/index.vue</code> ,<br><code>frontend/src/views/Room.vue</code>                                       | 负责房间入口、弹幕渲染、右侧四模块分析面板、OBS 状态订阅和联合分析触发。                              |
| Tornado 后端    | <code>main.py</code> , <code>api/*.py</code> , <code>services/*.py</code>                                                          | 集中注册 HTTP/WebSocket 路由，管理直播间连接、插件进程、OBS bridge 状态、AI 分析和直播归档。       |
| OBS Bridge 插件 | <code>obs-blivechat-bridge/src/obs-blivechat-bridge.cpp</code>                                                                     | 采集 OBS 推流事件、截图、麦克风/桌面音频，压成 JSON/WAV base64 后 POST 到本地后端。            |
| AI/Agent 层    | <code>api/qwen_multimodal.py</code> , <code>services/psych_agent.py</code> ,<br><code>agent_pipeline.py</code>                     | 把弹幕统计、画面帧和音频片段组装为多模态请求，调用 Gemma 4 系列模型输出结构化报告。                      |
| 插件生态          | <code>services/plugin.py</code> , <code>api/plugin.py</code> , <code>plugins/*</code>                                              | 通过子进程和 WebSocket 连接 text-to-speech、msg-logging、login、native-ui 等插件。 |
| 直播归档/复盘       | <code>services/live_archive.py</code> , <code>api/live_archive.py</code> ,<br><code>frontend/src/views/LiveReviewDetail.vue</code> | 将心理、语音、画面、礼物与直播元信息写入分文件归档，并生成封面、时间线和自动切片建议。                         |

三、Agent 设计

**实时 Room Agent**

- `Room.vue` 每个窗口聚合弹幕、礼物、SC、OBS 状态，调用 `/api/qwen/joint-report`，再分发到心理、语音、礼物、画面四个模块。
- `services/psych_agent.py` 是后端常驻心理状态服务，可从弹幕事件生成窗口 metrics，并调用 Qwen 或规则 fallback。
- `agent_pipeline.py` 是命令行离线/实验 Agent，连接 `/api/chat` 聚合窗口，调用 `/api/qwen/chat` 验证提示词和指标体系。
- Agent 输出协议包含 stress、valence、fatigue、engagement、confidence、danmakuDistill、liveEnvironment、voiceReport、sceneReport。

**输出分发**

- 心理模块：压力、积极度、疲劳、参与度、弹幕蒸馏。
- 语音模块：转写、声音类型、活动、外显状态。
- 礼物模块：礼物、舰长、SC、累计价值。
- 画面模块：游戏中、唱歌中、观看视频中及证据。

四、OBS 插件设计

obs-blivechat-bridge

- OBS 插件固定上报到 `http://127.0.0.1:12450`，避免跨机网络和认证复杂度。
- 事件通道上报 `streaming_started` / `streaming_stopped`、录制路径、推流时长。
- 画面通道上报最新截图或截图路径；后端再压缩为 JPEG data URL。
- 音频通道采样 16kHz 单声道 WAV base64，支持 mixed、mic、desktop 三种 `trackRole`。
- 后端 `obs_bridge` 保存最新状态，并通过 `/api/obs-agent` WebSocket 广播给前端。

设计取舍：OBS 插件只负责采集和上传，不做 AI 推理；后端统一保存最新状态并决定何时调用模型。这样可以把 OBS 端复杂度控制在 C++ 插件可维护范围内，同时让 Python/Vue 侧更容易迭代分析策略。

五、软件之间的通信

| 发送方/接收方                | 协议/接口                                                | 用途                                                      |
|------------------------|------------------------------------------------------|---------------------------------------------------------|
| 浏览器 Room               | WebSocket <code>/api/chat</code>                     | 接收弹幕、礼物、SC、心跳和错误事件。                                     |
| Room ↔ 后端              | HTTP <code>/api/qwen/joint-report</code>             | 发送窗口 metrics，换取心理/语音/画面联合报告。                            |
| OBS 插件 → 后端            | HTTP POST<br><code>/api/obs/event frame audio</code> | 推送推流事件、截图、音频片段和分轨配置。                                    |
| 后端 → Room              | WebSocket <code>/api/obs-agent</code>                | 广播 OBS state/event/frame/audio/config。                  |
| Home → Room<br>iframe  | <code>postMessage roomSetPsychTabConfig</code>       | 同步四模块启用状态和分析间隔。                                         |
| Room → Home            | <code>postMessage roomPsychAgentUpdate</code>        | 回传最新四模块结果给父页面。                                          |
| 后端 ↔ 插件                | WebSocket <code>/api/plugin/ws</code> + Bearer token | 插件心跳、命令广播、消息日志和 TTS。                                    |
| 后端 → Gemma<br>服务       | HTTP SSE <code>/v1/chat/completions</code>           | 流式读取 <code>delta.content</code> ，reasoning 只写 debug 日志。 |
| Room → live<br>archive | HTTP <code>/api/live/archive/*</code>                | 追加 mental/voice/picture/gift/live_info 记录，供复盘页读取。       |

六、直播归档与复盘链路

写入侧

- `Room.vue` 在四模块结果生成后调用 `/api/live/archive/append`。
- 分类文件包括 `mental.txt`、`voice.txt`、`picture.txt`、`gift.txt`、`live_info.txt`。
- 推流开始时通过 `/api/live/archive/init` 创建直播 session。

读取侧

- `LiveReviewDetail.vue` 读取 session、clip timeline、scene timeline。
- `services/live_archive.py` 解析归档文本，生成场景时间轴和高光切片建议。
- 可用 `ffmpeg` / `ffprobe` 抽帧生成封面、读取录播时长。

七、客户端使用的 Tools 与库

| 工具/库                                             | 用途                             |
|--------------------------------------------------|--------------------------------|
| Vue 2 + Element UI                               | 客户端 UI、房间配置、卡片/开关/表单。          |
| vue-router / vue-i18n                            | 页面路由和中日英文案。                    |
| axios + fetch + WebSocket                        | HTTP API、SSE 间接消费、弹幕/OBS 实时通信。 |
| Tornado                                          | Python HTTP/WebSocket 服务器。     |
| aiohttp / httpx / openai SDK                     | 外部 API、Qwen/Gemma 调用和带宽诊断。     |
| blivedm                                          | B 站直播弹幕连接与消息模型。                |
| OBS C++ API + libcurl                            | OBS 插件、截图/音频采集、HTTP 上传。        |
| Pillow                                           | 截图压缩为 JPEG，降低 :8443 上传压力。      |
| SQLAlchemy                                       | 数据持久化基础设施。                     |
| downloadjs / echarts / crypto-js / pako / lodash | 前端下载、图表、哈希、压缩和工具函数。            |

八、当前架构的关键质量措施

**上传链路诊断**

api/qwen\_multimodal.py 记录 body 大小、首 SSE 字节、首文本 token、content/reasoning chunk 数量，并在上传错误中计算近似上传网速。

**非 JSON 容错**

Room.vue 会尝试从响应文本中提取 JSON；解析失败时只显示中文规则兜底，不展示 reasoning 原文。

**插件隔离**

插件由子进程启动，并通过 Bearer token + WebSocket 接入；插件异常不会直接拖垮主 Tornado 服务。

九、建议

- 1. 把 /api/qwen/joint-report 的输出 schema 固化为版本化协议，例如 jointReport.v1，并在前后端共享字段约束。
- 2. 将 OBS 上传链路从 AutoDL :8443 网关逐步迁移到更稳定的本地隧道或对象存储中转，减少 0.5MB 级请求的抖动。
- 3. 将 Room.vue 中 Agent 编排逻辑拆到独立 composable/service，降低单文件复杂度。
- 4. 为 content/reasoning、非 JSON、空响应、413、ReadError 等路径补充自动化回归测试。